

S6 explained — skills

A full walkthrough of what was built, why, and what to say when asked.

1. What a skill is

The brief sets one bar, and the last clause is the whole design constraint:

Implement **at least 2 skills**. They must provide real student value, extend the LLM's capabilities, and **not be renamed versions of the same prompt**.

That sentence is a warning about the obvious implementation. Write `prompts/quiz_builder/v1.md`, call the model, save whatever comes back, and you have a "skill" that is a prompt with a route in front of it. It will demo fine. It is also exactly what the brief says does not count.

So a skill here is defined as **a prompt plus a piece of code that checks the model's work**:

```
generate(...) -> ask the model
                -> run the skill's own checks, in Python
                -> problems? name them, ask again
                -> still broken? fail loudly rather than serve it
```

Take the checks away and each one really would be a renamed prompt. With them:

- `step_solver` produces algebra a computer algebra system has **agreed with**, line by line.
- `quiz_builder` produces a quiz whose options are **known** to be distinct and whose correct answers are **known** not to sit in one slot.

Neither property is obtainable by asking more nicely. That is the test for whether a skill is real: *name the property the code guarantees that the prompt cannot*.

Skill vs tool

S4 built tools. They look similar and are not the same shape:

	Tool	Skill
Who invokes it	the model, mid-sentence	the student, explicitly
Size	a few hundred tokens	a whole piece of work
Output	text back into the prompt	a structured result worth keeping
Checked by	argument validation	its own code, before storage

Tool	Skill
Leaves behind	a <code>tool_calls</code> row
	a <code>skill_runs</code> row and a turn

A tool answers a question the model asked itself. A skill is a job the student asked for.

2. The shape of the solution

```

POST /rooms/{id}/skills/quiz_builder/runs {"topic": "...",
"question_count": 5}
  |
  |├─ resolve      dict lookup in the registry      -> 404 if unknown
  |├─ validate     against the skill's own args_model -> 422, nothing
  |
  |attempted
  |├─ Turn         status=running, seq = next in room
  |├─ SkillRun     turn_id, status, request
  |
  |├─ skill.run()
  |   |├─ search the room's documents for the topic (grounding)
  |   |├─ generate()
  |   |   |├─ call the model      -> turn_attempts row,
  |   |   |
  |   |   |purpose='skill'
  |   |   |├─ parse + schema + checks -> problems, written for the
  |   |   |
  |   |   |model
  |   |   |├─ problems? repair      -> another turn_attempts row
  |   |   |├─ clean? return
  |   |   |
  |   |   |before
  |   |   |├─ balance_positions()  shuffle, seeded – after the checks, never
  |   |   |├─ quiz_questions / quiz_choices rows  the database enforces two
  |   |   |├─ rules
  |   |   |   └─ Turn -> succeeded, answer_text, concepts
  |   |
  |   |201 { "turn_id": ..., "status": "succeeded", "questions": [...],
  |   |"checks": [...] }
  |   |   |
  |   |   |└─> GET /turns/{turn_id}  prompts, raw replies, tokens,
  |   |
  |   |failures

```

The key idea: nothing reaches the student that our own code has not agreed with, and the evidence of that agreement is stored, not logged.

3. A skill run is a turn

`skill_runs.turn_id` is `UNIQUE NOT NULL`. Asking for a quiz is asking the room a question — the fact that the answer is five structured questions rather than a paragraph does not make it a different kind of event.

Four things follow, and together they are why this shape was chosen:

- **One audit trail.** The model calls a skill makes are `turn_attempts` rows with `purpose='skill'`, alongside the tutor's and the repair loop's. So `GET /turns/{id}` explains a quiz in exactly the detail the brief asks for — prompt, attempts, failures, tokens — with no second inspection format.
- **One timeline.** A quiz built in a room appears in that room's turn list, where the student asked for it.
- **It counts as studying.** Study history reads succeeded, in-scope turns, so a quiz on photosynthesis contributes its concepts like any other turn.
- **It carries context.** The tutor sees the quiz request in the last six turns, so "explain question 3" is answerable.

The version that was built first and reversed

The first attempt hung `skill_runs` off `rooms`, with the model attempts in a JSONB column on the run. It worked. It was reversed before being committed, for one reason:

It left the project with **two audit trails of different shapes**. "Inspect the model attempts" meant one thing for a chat and another for a quiz, and the room's conversation had a hole in it where the quiz was.

`purpose='skill'` has been in the `AttemptPurpose` enum since S2 — the S2 design already expected this, and the first implementation ignored it. Worth saying on the call: the plan was right and the first pass was wrong, which is what a plan is for.

The turn still has to be a complete turn

`terminal_states_are_complete` (S2) requires a succeeded turn to carry `answer_text` and `in_scope`. That is not a constraint to route around — it is the reason every skill implements `summarise()`:

```
"Here is a 5-question quiz on photosynthesis, 3 of them from your uploaded material."
```

That is what belongs in a timeline. The quiz itself is one link away.

4. `quiz_builder` — four failures, four different places

The brief names exactly what a bad quiz looks like:

a quiz might contain **duplicate choices**, the **wrong count**, an **invalid answer**, or **predictable answer option patterns**

All four are handled. **Where** each one is handled is the interesting part, and it is the answer to "how is this not a prompt":

Failure	Caught by	Why there
duplicate choices	code, then <code>ux_choice_text</code>	code names it for a retry; the index makes it unstorable
wrong count	code only	a fact about a set of rows — a constraint would need a trigger
invalid answer	code, then <code>ux_choice_correct</code>	same pair of reasons as duplicates
predictable positions	code alone, by shuffling	see below

The two indexes:

```
CREATE UNIQUE INDEX ux_choice_text
ON quiz_choices (question_id, lower(btrim(text)));

CREATE UNIQUE INDEX ux_choice_correct
ON quiz_choices (question_id) WHERE is_correct;
```

The first means two options in one question cannot say the same thing, with `"Paris"`, `" paris "` and `"PARIS"` counted as one answer — which is how a student reads them. The second means a question cannot have two right answers; the partial `WHERE` is what makes it work, because without it a question could hold only one *wrong* option too.

Both were verified by inserting bad rows by hand rather than assumed from the migration:

```
ERROR: duplicate key value violates unique constraint "ux_choice_text"
ERROR: duplicate key value violates unique constraint "ux_choice_correct"
```

Why code *and* a constraint for two of them. They do different jobs. The code produces a sentence the model can act on — *"question 1 options 1 and 2 are the same answer written twice ('paris')"* — which is what makes the repair work. The index produces nothing readable and cannot be forgotten by a future code path. Reporting and preventing are not the same task.

Every problem is reported, not just the first

```
def check_quiz(quiz: DraftQuiz, *, expected: int) -> list[str]:
```

One repair call fixing four faults beats four calls fixing one each. The list is written **for the model to read**, which is why it names positions and quotes text rather than saying `ValidationError`.

The failure that cannot be a check

Predictable answer positions is the fourth named failure and the only one that is not a property of any single quiz. A quiz with the answer in slot B is not wrong. Models put it there far more often than chance, and instructing them not to does not work — they are not disobeying an instruction, it is a bias in what they generate.

So it is fixed by construction, after the checks pass:

```
def balance_positions(quiz: DraftQuiz, *, seed: int) -> DraftQuiz:
    rng = random.Random(seed)
    ...
```

The prompt even says so, which removes the model's incentive to try:

Do not try to vary which position the correct answer sits in — the options are shuffled after you reply, so any pattern you produce is removed.

Shuffling happens **after** the checks, never before: shuffling a quiz that is about to be rejected wastes the work, and the checks do not care about order.

Honest limit, worth volunteering. An independent shuffle per question spreads the answer evenly *in expectation*, not by construction. Five questions can all land on B about once in a thousand runs. Making the claim exact would mean assigning slots round-robin from a seeded permutation rather than shuffling each question alone. The test asserts the spread across twenty questions, which is the property that actually matters, but the docstring's "true by construction" is stronger than the code.

Why the seed comes from `hashlib`

```
digest = hashlib.sha256(f"{topic}|{args.question_count}".encode()).digest()
return int.from_bytes(digest[:4], "big")
```

Not the built-in `hash()`. Python randomises string hashing per process unless `PYTHONHASHSEED` is set, so `hash()` would be stable *within* a run and different on the next one — which is exactly the failure the fixed seed exists to prevent, and it would have shown up as a test that passes until it does not.

Grounding

If the room has uploaded material, the quiz is built from it: the topic is searched with S5's `document_service.search`, the passages go into the prompt with their citations, and each question records the `cite` it came from. A quiz drawn from the student's own handout tests their course rather than the subject in general.

Also an honest limit. Nothing verifies that `source` matches a citation that was actually offered — the model writes it, and `summarise()` then tells the student "3 of them from your uploaded material". A set-membership test against the passages handed to the prompt would make that true rather than claimed. It is in the README's "what I would change".

5. `step_solver` — what "correct" means for a step

The brief's own example of a skill is *"a step-by-step math explainer"*, and it is a good example precisely because the naive version is a renamed prompt: ask the model to explain step by step and it will, fluently, including the times it drops a sign.

The rule

A solving step rewrites an equation without changing its solutions:

$$3x + 6 = 15 \quad \rightarrow \quad 3x = 9 \quad \rightarrow \quad x = 3$$

Each of those has exactly one solution, $x = 3$. So the check is **not** "does this line look like the last one with 6 subtracted" — it is **do these two equations have the same solution set**:

```
def steps_are_equivalent(before: str, after: str) -> StepCheck:
    ...
    left, right = _solutions(first), _solutions(second)
    if left == right:
        return StepCheck(True, "")
```

That single rule catches every arithmetic slip without knowing which operation the model claimed to perform. **A step is accepted because it preserves the answer, not because its explanation sounded reasonable** — which is what makes it hard to talk past. The parametrised test includes $2(x - 4) = 10 \rightarrow 2x - 4 = 10$ (forgot to distribute) and $x^2 - 4 = 0 \rightarrow x = 2$ (dropped a root); both are rejected, and neither could be caught by reading the prose.

The final answer is checked separately, by substituting it back into the original problem. Worth doing even when every step passed: the steps prove the model transformed the equation faithfully, and this proves the thing it ended on is

actually a solution to what was asked. A chain can be individually valid and still not finish the job.

When a step cannot be checked

Not every line is decidable. `StepCheck` carries a third field for exactly this:

```
@dataclass(frozen=True)
class StepCheck:
    ok: bool
    detail: str
    checkable: bool = True
```

Callers gate on `checkable` before reading `ok`, and **both directions matter**: reporting an undecidable step as a failure would reject working that is probably fine, and counting it as verified would claim a machine agreed with a line it never looked at. `verified_steps()` stores both flags on the run, so the inspection endpoint shows exactly which lines were confirmed:

```
{"position": 1, "expression": "3x = 9", "verified": true, "checkable": true}
```

"We verify the maths" is a claim. That is the evidence.

The skill declines what it cannot verify

```
try:
    equation = parse_equation(problem)
except UnparseableStep as exc:
    raise SkillFailure("UNCHECKABLE_PROBLEM", ...)
```

Before spending a model call. If the checker cannot read the problem it cannot verify any step of the answer, and **this skill without its checks is the renamed prompt the brief warns about** — while the tutor endpoint already answers word problems perfectly well. Declining is the stronger behaviour, and it is one line of reasoning you can defend in a sentence.

6. `verify.py` — the file most likely to be attacked

It takes a string that originates with a student and passes through a language model, and hands it to a computer algebra system. That is the most dangerous shape a function can have, and it is the S6 equivalent of S4's calculator.

`sympify` is not safe

`sympy.sympify` runs Python and has had sandbox escapes. Same posture as `evaluate_expression`: an allow-list first, then the library.

```
_ACCEPTABLE = re.compile(r"^[0-9a-zA-Z+\-*/^().=\s]+$")
```

Digits, letters, four operators, `^`, brackets, `=`, `.`. No quotes, no underscores, no dot followed by a name — so no attribute access and no dunder. A string that fails it never reaches sympy at all. Three hostile inputs are tested, including `__import__("os").system("ls")` and `x.__class__.__bases__`.

The allow-list stops code, not cost

This is the part worth leading with, because it is the same lesson S4 learned in a different place and it was **not** carried over when this file was first written.

```
9^9^9 = 1
```

Six characters. Every one of them on the allow-list. One `=`. Under the length limit. And `parse_expr(evaluate=True)` asks Python for a 370-million-digit integer — on the event loop, with nothing able to interrupt it, in a request someone is waiting on. Measured: it had not returned after 20 seconds.

Its symbolic cousin is worse, because it parses instantly and then hangs later:

```
x^99999 = 1      parses in ~1 ms, then never leaves solveset
```

Neither is a sandbox escape. Both are a hang, which for a synchronous API is the same outcome. **A denial of service reachable with one request and zero model spend.**

The fix: look before evaluating

```
tree = parse_expr(cleaned, transformations=_TRANSFORMS, evaluate=False)
_reject_runaway_powers(tree)
return parse_expr(cleaned, transformations=_TRANSFORMS, evaluate=True)
```

Parsed twice on purpose. `evaluate=False` builds `9**9**9` as three nodes in microseconds, where evaluating it first would already have hung. So the guard gets to look before any arithmetic happens.

An exponent that is itself a calculation is refused rather than measured — the same rule, for the same reason, as `_check_power` in the S4 calculator: you cannot check

the size of 9^9 without computing it, and computing it is the thing being guarded against.

Two limits, because there are two costs

```
MAX_NUMERIC_EXPONENT = 1000      # a number to a number: cost is digits
MAX_POLYNOMIAL_DEGREE = 20       # a variable to a number: cost is solveset
```

One number could not express both. 2^{1000} is a 302-digit integer and instant.
 x^{100} is a hundred characters and takes fifteen seconds. Measured:

degree	solveset
4	0.02s
20	0.42s
50	2.19s
100	14.60s

School algebra does not go past degree four, so 20 is already generous. The limits are set from measurement, not from taste — which is the answer to "how did you pick those numbers".

Results, through the live API:

```
9^9^9 = 1      -> UNCHECKABLE_PROBLEM in 675ms  (was: never returned)
x^99999 = 1    -> UNCHECKABLE_PROBLEM in 43ms   (was: never returned)
```

Parsing is not proof of maths

The second thing the allow-list does not decide. This passes every test above:

```
"Give me a quiz on TCP"
```

Letters and spaces only, so the regex admits it, and implicit multiplication turns it into $C*G*P*T*a*e**2*i**2*m*n*o*q*u*v*z$ — a perfectly valid expression that is not a problem anybody asked to solve. Rejection was being driven by **punctuation**:
"What is photosynthesis?" was refused for its question mark, not for being prose.

Symbol count separates them cleanly:

free symbols	input
1	$3x + 6 = 15$

free symbols	input
1	$2(x - 4) = 10$
2	$x + y = 2$
9	John has 3 apples
14	Give me a quiz on TCP

MAX_UNKNOWNNS = 3

And the bound is not arbitrary: above **one** unknown `_solutions` already returns `None` and the checker drops to a weaker fallback, so a problem with fourteen symbols was never going to be verified. Refusing it says out loud what was already happening quietly — and it saves three model calls that used to end in `CHECKS_FAILED`, which is the wrong reason as well as the slow one.

The cap lives in `StepSolverSkill.run`, deliberately **not** in `parse_equation`, so the per-step checking is unaffected.

7. The repair loop — three budgets, kept apart

`skills/base.generate` is shared by both skills. It runs three counters, and they count three different problems:

Budget	What it counts	Default	Why separate
<code>max_network_retries</code>	provider errors	2	nothing was wrong with the prompt
<code>max_repair_attempts</code>	unusable answers	2	the prompt has to change
<code>skill_deadline_seconds</code>	the whole run	90	bounds the sum

Collapsing the first two would mean **three rate limits ate the repair budget**, and the model never gets its second chance at the thing it actually got wrong. Both counters reuse the tutor's settings rather than adding new ones.

`llm_request_timeout_seconds` (30) is set on the provider client and is the only thing that can end a request that *hangs* — the deadline is checked between attempts, and a call that never returns never reaches the check. Two bounds, because they bound different failures.

Failed calls write their attempt row **before** the retry decision, so a retried call still leaves its own row. A retry hidden inside a wrapper writes nothing, and then the inspection endpoint under-reports what the turn cost.

Failures keep the provider's own name — `EMPTY_RESPONSE`, `RESPONSE_BLOCKED` — rather than collapsing to `PROVIDER_UNAVAILABLE`, because `TurnFailureReason`

already carries those three values to mirror `LLMError.error_type` exactly. When they were collapsed, a turn and the attempt row beneath it disagreed inside one response.

What it still does worse than the tutor

Be the one to raise this. `prompts/tutor_repair/v1.md` is a separate versioned file that shows the model its own rejected output between markers, labelled as data rather than instruction. A skill instead pastes a Python string constant onto the end of the original prompt:

```
Your previous quiz was rejected by automatic checks. Fix exactly these problems:  
- question 1 options 1 and 2 are the same answer written twice ('paris')
```

The model is told what to fix **without being shown the thing to fix**. It regenerates rather than repairs, and mostly gets away with it. Two consequences:

1. `prompt_sha256` on a repair attempt describes only the base file, so the checksum no longer identifies the text that was sent. `rendered_prompt` still holds the truth, so nothing is lost — but a column that claims something false is worse than one that is absent.
2. `docs/PLAN.md` §11 lists `quiz_repair` as a prompt to write. It shipped as a constant.

The fix is two prompt files and threading `previous_output` through `generate`, which is a shape the tutor already proves works.

8. Why the quiz gets tables and the solver gets JSONB

The asymmetry is deliberate and it is a good question to be asked.

Two of the brief's four quiz failures are things a **database can make impossible to store**, and the two unique indexes do exactly that. So the quiz gets real tables, because there are real constraints to hang on them.

`step_solver` has no equivalent. No constraint can express "this algebra step follows from the last one" — that is sympy's job, at write time, before the row exists. Giving its steps three more tables would buy nothing a `JSONB` column does not, so `skill_runs.result` holds them.

Structure where the database can enforce something; documents where it cannot.

There is also no `quizzes` table between `skill_runs` and `quiz_questions`. A run produces at most one quiz, so that row would have carried nothing but a second id. `quiz_questions.run_id` points straight at the run.

Same reasoning killed the second identifier on the API. A run is 1:1 with its turn, so a response carrying both `id` and `turn_id` invited reaching for `id` — and `GET /turns/{id}` would then answer "no turn with that id", which reads as the feature being broken. The run's primary key stays internal and every path takes a turn id.

9. The API surface, and what is wrong with it

Four routes:

```
POST /rooms/{id}/skills/quiz_builder/runs
POST /rooms/{id}/skills/step_solver/runs
GET  /rooms/{id}/skills/runs                cursor-paged summaries
GET  /turns/{id}/skill-run
```

The first version took the skill name as a path parameter and the arguments as an untyped `dict`. It cost what you would expect: Swagger could not show that `quiz_builder` takes `topic` and `question_count` while `step_solver` takes `problem`, so the single example on the shared endpoint was wrong for whichever skill you were not looking at — and a misleading example is worse than none.

Typed routes give each skill its own request schema and example, generated from the `args_model` the skill already declares. The registry still owns *which* skills exist, so there is still no path from input to arbitrary code.

And it is still the wrong shape. Say this before it is pointed out: the project's own thesis is "a skill run is a turn", and the API contradicts it — a turn is created at `/rooms/{id}/turns` unless it is a skill turn, in which case it is somewhere else. The right surface is one endpoint with a body discriminated on `skill` and a response discriminated on `content.type`:

```
POST /rooms/{id}/turns
{ "skill": "quiz_builder", "topic": "TCP", "question_count": 5 }
```

That gets the per-skill Swagger documentation *and* one entry point, and it replaces `result: dict | None` with a typed shape. Not built because it is a surface refactor with no new capability, and it would have meant rewriting a passing test file on the deadline. Written up in the README instead.

Failure is a 201

A failed run returns `201` with `status: "failed"`, matching how turns already behave. The run keeps `failure_reason`, `failure_detail`, and every check that rejected each attempt.

CHECKS_FAILED · UNCHECKABLE_PROBLEM · DEADLINE_EXCEEDED
EMPTY_RESPONSE · RESPONSE_BLOCKED · PROVIDER_UNAVAILABLE

A quiz that could not be built is the most informative row in the table. The attempts hold every prompt and every reply; **checks** holds exactly which rule rejected each one. That is the evidence for "we do not show broken output" — deleting it would leave only the claim.

The plan originally said a failing quiz should have its bad questions dropped. That was not built, on purpose: the checks are per-question, so dropping the failures leaves a quiz of the wrong length, which is **itself one of the four named failures**. Failing whole is the correct behaviour, and it is a better answer than the plan's.

10. Real bugs found while building

Every one was found by running the system or deliberately attacking it, not by reading it.

Two audit trails of different shapes

Covered in §3. Built, reversed before committing, and the enum value the correct design needed had been sitting in the codebase since S2.

9⁹ never returned

Covered in §6. The most serious defect in the slice: one request, no API key spend, and the event loop is gone for the whole process. Found by probing the allow-list with inputs that satisfy it rather than inputs that violate it — which is the habit worth describing, because "what passes my check and is still bad" is a different question from "what does my check catch".

An answer with no sides

```
answer_satisfies("2x = 4", "2 = 2")
-> AttributeError: 'BooleanTrue' object has no attribute 'lhs'
```

sympy evaluates `Eq(2, 2)` to a plain boolean, which has no **lhs** to read. So an answer stating an identity rather than a value crashed the checker — an **AttributeError** becoming a 500, from the one module whose entire job is to fail in the open. Same shape for `x = x`.

Fixed with a type check before the attribute access, and the outcome is now **checkable=False** rather than a failure, because we reached no verdict.

Two functions disagreeing about what "unverifiable" meant

`check_solution` and `verified_steps` both decided whether a step was checkable by **substring-matching the error message**:

```
if not outcome.ok and "does not accept" not in outcome.detail:
```

Two callers sniffing prose from a third module, and they disagreed: one treated **"could not read"** as not-checkable, the other treated it as a failure. So the module docstring's promise that unverifiable steps are not failures held only for one of the two paths, and editing a message would silently change what the solver rejected. Replaced with the `checkable` flag on `StepCheck`.

Prose that parses

Covered in §6. The gate was rejecting on punctuation and calling it maths detection.

The reliability layer skills never got

`skills/base.generate` caught `LLMError` and failed immediately. The tutor has retried (`ProviderUnavailable`, `EmptyResponse`) with backoff since S3.

The consequence was specific rather than theoretical: `turn.py`'s own comment notes that an empty response is *common* on `gemini-2.5-flash`, because thinking tokens count against the output budget and how much it thinks varies run to run. So the likeliest real failure in the project killed a quiz run outright while the tutor beside it shrugged and retried. `RETRIABLE` and `backoff` moved to `app/llm/retry.py`, beside the errors they name, and both loops import them.

A relationship SQLAlchemy quietly dropped

Assigning `SkillRun(turn=turn)` on an object not yet in the session leaves the cascade with nothing to follow, and the skill's first attempt triggers an autoflush immediately afterwards. SQLAlchemy warns and drops the association. Building the row with `turn_id=turn.id` and adding it to the session first fixes it.

`text` shadowing `text()`

`QuizChoice` has a column called `text`, which shadows SQLAlchemy's `text()` function inside the class body and turns the functional index expression into **"MappedColumn object is not callable"**. Imported as `sql_text` instead. A small thing, but it is the kind of error message that costs twenty minutes if you have not seen it.

A list endpoint that was a download

`GET /rooms/{id}/skills/runs` originally returned full `SkillRun` objects. One ten-question quiz is about 10 KB, and every question and option was read from the database to be discarded by the serialiser. Replaced with a `RunSummary` and a `COUNT` subquery — the questions are never loaded.

Paged on the turn's `seq`, not `created_at`: `seq` is unique per room and strictly increasing, so it cannot tie the way two runs in the same second can. That is the same lesson S5 learned when eviction ordered by a timestamp that was identical across a transaction.

11. Files

New:

<code>app/skills/base.py</code> (336)	what a skill is; the shared generate loop
<code>app/skills/registry.py</code> (33)	the fixed list
<code>app/skills/quiz_builder.py</code> (246)	checks, balancer, grounding
<code>app/skills/step_solver.py</code> (207)	sympy checks, the unknowns gate
<code>app/skills/verify.py</code> (270)	equivalence, substitution, the guards
<code>app/services/skill.py</code> (300)	run, fail, get, list
<code>app/models/skill_run.py</code> (231)	<code>skill_runs</code> + <code>quiz_questions</code> + <code>quiz_choices</code>
<code>app/schemas/skill.py</code> (110)	summary and full-run shapes
<code>app/api/routes/skills.py</code> (154)	four typed routes
<code>app/llm/retry.py</code> (47)	RETRIABLE + backoff, shared with the tutor
<code>prompts/quiz_builder/v1.md</code> (88)	
<code>prompts/step_solver/v1.md</code> (57)	
<code>migrations/versions/20260814_1634_skill_runs_on_turns.py</code> (178)	
<code>tests/test_skills.py</code> (839)	56 tests

Changed:

<code>app/models/turn.py</code>	the 1:1 <code>skill_run</code> relationship
<code>app/schemas/turn.py</code>	<code>skill_run</code> nested under <code>TurnDetail</code>
<code>app/services/turn.py</code>	<code>next_seq</code> made public; eager-loads the run; uses

llm/retry	
app/config.py	llm_request_timeout_seconds,
skill_deadline_seconds	
app/llm/gemini.py	a timeout on the client (milliseconds – the SDK's unit)
app/main.py	the skills tag and routers
tests/test_prompts.py	both new prompts under the pinned-version guard

290 tests pass. Ruff clean. Migration round-trips with no drift.

12. Questions you should expect

"How is your skill not just a prompt?"

The single most likely question, and the whole slice is the answer. Name the property the code guarantees: `step_solver`'s steps have been agreed with by a computer algebra system, one line at a time, and a failing line is sent back. Then show `verified: true` on the run. Then the quiz: two of the four named failures are *unstorable*, not merely checked, and the fourth is fixed by shuffling because it cannot be checked at all.

"Walk me through the four quiz failures."

Use the table in §4 and lead with *where* each is handled, not that all four are. The interesting content is why duplicates get both code and an index, why wrong count cannot be a constraint without a trigger, and why predictable positions cannot be a check at all.

"What stops the model from talking its way past your maths checker?"

It never sees the check. Equivalence is decided on solution sets, so the checker knows nothing about which operation was claimed — a step is accepted because it preserves the answer. Give the distribute example.

"Is handing model output to sympy safe?"

Two separate answers, and give both. Code: an allow-list runs first, `sympify` is never called on raw input, three hostile inputs are tested. Cost: the allow-list does not help there, and `9^9^9` hung the process until the power guard was added. That second half is the better answer, because it shows the limit of the first.

"How did you pick 1000 and 20?"

By measuring. Degree 100 takes 14.6 seconds in `solveset`; degree 20 takes 0.4. A numeric power is bounded by digits, a symbolic one by polynomial degree, so one limit could not serve both.

"Why does a skill run create a turn?"

Because a quiz *is* a turn, and the alternative gave the project two audit trails. Then the four consequences in §3. Mention that the first implementation got it wrong and was reversed before it was committed.

"What happens when the model cannot produce a valid quiz?"

The run fails with `CHECKS_FAILED` and keeps every attempt and every verdict. Then volunteer the plan change: dropping bad questions would leave a quiz of the wrong length, which is one of the four named failures, so failing whole is correct.

"What's the weakest part of this?"

Three, in order. The repair prompt does not show the model its own rejected output, so it regenerates rather than repairs — and the tutor already does this properly in the same repo, which makes it a gap rather than an unknown. `source` is an unverified claim the summary repeats to the student. And the API should be one endpoint with a discriminated body, not four routes, because the current shape contradicts "a skill run is a turn" at exactly the place a reviewer looks first.

"How did you find these problems?"

By attacking the allow-list with inputs that satisfy it, rather than inputs that break it — `9^9^9` passes every character test and every length test and still takes the process down. And by asking what the tutor does that the skills do not, which is how the missing retry surfaced. Both are questions about the gap between what a check says and what it covers.